

FLIGHT PROCEDURES MANUAL

PROJECT ZEPHYRUS

J. BOLTE, C. CASEBOLT, M. NICHITIU,
S. PARKE

MIT ROCKET TEAM AVIONICS

LAST UPDATED: FEB 17 2026



Instructions

This manual is for use on Project Zephyrus flights, to be printed for launch in a minimum of 2 copies, one for the Telemetry Lead, and another for the Telemetry Deputy. It is to be read prior to launch by all members involved in the Ground Station, Away Station, and Avionics/Structures Lead roles. At each of the Away Station(s) and Ground Station, 2 copies must be printed and be available for the site lead and deputy to access prior to, during, and after flight. The personnel whose mastery of this document's contents is listed below:

- **Team Lead**
(Sections 1-9)
- **Ground Station Lead**
(Sections 1-2, 4-6, 8)
- **Away Station Lead**
(Sections 3-6, 9)
- **Avionics Lead**
(Sections 1-9)
- **Ground Station Deputy**
(Sections 1-2, 4-6)
- **Away Station Deputy**
(Sections 3-6, 9)
- **Structures Lead**
(Sections 1, 4-6)
- **Simulation Lead**
(Sections 1-11)
- **Integration Lead**
(Sections 1-9)

Contents

1	Avionics Setup Instructions	2	6	Post-Flight Procedures	7
1.1	Avionics Bay	2	7	Onboard Electronics Documentation .	8
1.2	Fin Control	2	7.1	Onboard Hardware Documentation	8
1.3	Airbrakes Module	2	7.2	Onboard Software Documentation	8
1.4	Inter-Subsystem Connections	2	8	Ground Station Documentation . . .	9
2	Ground Station Setup Instructions .	3	8.1	Ground Station Hardware . . .	9
2.1	Telemetry Reception	3	8.2	Ground Station Software . . .	9
2.2	Live Video Reception	3	9	Away Station Documentation	10
3	Away Station Setup Instructions . .	4	9.1	Away Station Hardware	10
3.1	Antenna Pointer Assembly . .	4	9.2	Away Station Software	10
3.2	Antenna Pointer Setup	4	10	FC Simulation Documentation	11
4	Pre-Flight Procedures	5	10.1	Usage Instructions	11
4.1	Go-No Go Criteria	5	10.2	Usage Examples	11
5	In-Flight Procedures	6	10.3	FC Simulation Framework	11
5.1	Expected Behavior	6	10.4	OR Simulation Framework	11
5.2	Flight Detection Failure Procedure	6	10.5	Python Scripting Framework .	11
5.3	Flight Computer Reset Procedure	6	11	Airbrakes Control Algorithm	12
5.4	Lost Connection Procedure . .	6	11.1	Open Loop: Feedforward . . .	12
			11.2	Closed Loop: Feedback . . .	13
			11.3	Onboard Algorithm	16
			12	Full FC Code Printout	21

1 Avionics Setup Instructions

1.1 Avionics Bay

1.2 Fin Control

1.3 Airbrakes Module

1.4 Inter-Subsystem Connections

2 Ground Station Setup Instructions

2.1 Telemetry Reception

2.2 Live Video Reception

3 Away Station Setup Instructions

3.1 Antenna Pointer Assembly

3.2 Antenna Pointer Setup

4 Pre-Flight Procedures

4.1 Go-No Go Criteria

5 In-Flight Procedures

5.1 Expected Behavior

5.2 Flight Detection Failure Procedure

TLDR advance state manually

5.3 Flight Computer Reset Procedure

TLDR assure visual apogee +1 second, arm and fire pyros manually; enter APOGEE state/DISREEF_AWAIT state manually

5.4 Lost Connection Procedure

TLDR cooked, antenna proc maybe

6 Post-Flight Procedures

7 Onboard Electronics Documentation

7.1 Onboard Hardware Documentation

7.2 Onboard Software Documentation

8 Ground Station Documentation

8.1 Ground Station Hardware

8.2 Ground Station Software

9 Away Station Documentation

9.1 Away Station Hardware

9.2 Away Station Software

10 FC Simulation Documentation

10.1 Usage Instructions

10.2 Usage Examples

10.3 FC Simulation Framework

10.4 OR Simulation Framework

10.5 Python Scripting Framework

Thus, we solve

$$A_0 = \frac{2mg(\Delta x)}{C_D \rho} \left(\int_{t_0}^{t_{\text{apog}}} ((a(t - t_{\text{apog}})^3 + b(t - t_{\text{apog}})^2 - g(t - t_{\text{apog}}))^3 dt \right)^{-1} \quad (11.11)$$

The integral expression gives

$$\begin{aligned} \int_{t_0}^{t_1} \dot{x}^3 dt = & - \left(\frac{a^3}{10} (t_0 - t_1)^{10} + \frac{a^2 b}{3} (t_0 - t_1)^9 + \frac{3ab^2 - 3a^2 g}{8} (t_0 - t_1)^8 \right. \\ & \left. + \frac{b^3 - 6abg}{7} (t_0 - t_1)^7 + \frac{ag^2 - b^2 g}{2} (t_0 - t_1)^6 + \frac{3bg^2}{5} (t_0 - t_1)^5 - \frac{g^3}{4} (t_0 - t_1)^4 \right) \end{aligned} \quad (11.12)$$

and so if $T \equiv t_{\text{apog}} - t_0$,

$$A_0 = f \cdot \left(-\frac{2mg(\Delta x)}{C_D \rho} \cdot \frac{T^{-4}}{\frac{a^3}{10} T^6 + \frac{a^2 b}{3} T^5 + \frac{3ab^2 - 3a^2 g}{8} T^4 + \frac{b^3 - 6abg}{7} T^3 + \frac{ag^2 - b^2 g}{2} T^2 + \frac{3bg^2}{5} T - \frac{g^3}{4}} \right) \quad (11.13)$$

where f is the aforementioned “fudge factor,” whose value is generally close to 3, tuned prior to launch. The latest values appear in the table below:

m (kg)	f for $\Delta x < 40$ m	f for $\Delta x \geq 40$ m
51.71	4.22	4.7

Table 1. “Fudge Factors” for Airbrakes Algorithm (AS OF FEB 15 2026)

11.2 Closed Loop: Feedback

We have decided that due to uncertainty in the true performance of the Airbrakes hardware, we include a feedback component. The main prerequisite for such a component is accurate projected altitude estimation.

Using a similar energy-based calculation, we assume that the only work done in rocket vertical motion arises from drag. Take $\hat{x}$ to be the upwards direction. The force balance thus gives

$$m\ddot{x} = -mg - \left(\frac{\rho C_{D_{\text{airbrakes}}} A(t)}{2} + c \right) \dot{x}^2 \quad (11.14)$$

where c is the drag magnitude of the rocket divided by $\dot{x}^2$:

$$c = \frac{F_{D_{\text{rocket}}}}{\dot{x}^2} = \frac{\rho C_{D_{\text{rocket}}} A_{\text{ref}}}{2} \quad (11.15)$$

This a constant and can be evaluated prior to flight. From the balance, we have that the work done by the total drag is

$$W_{\text{drag}} = \int_{h_0}^{h_f} -F_D dx \quad (11.16)$$

Since $\dot{x} \neq 0$ in the open interval of integration (only at the bound), we can perform the following change of variables:

$$\ddot{x} = \frac{d\dot{x}}{dt} = \frac{d\dot{x}}{dx} \frac{dx}{dt} = \frac{d\dot{x}}{dx} \dot{x} \Rightarrow dx = \frac{\dot{x}}{\ddot{x}} d\dot{x} \quad (11.17)$$

This allows us to write the energy conservation equation below, for some initial state $\{x = h_0; \dot{x} = v_0\}$ to the final state $\{x = h_f, \dot{x} = 0\}$:

$$mgh_f = \frac{1}{2}mv_0^2 + mgh_0 + \int_{v_0}^0 \left(-F_D \frac{\dot{x}}{\ddot{x}} \right) d\dot{x} \quad (11.18)$$

From the force balance in (11.14) and the drag force expression, we have that the integral becomes a functional depending on the airbrakes area profile $A(t)$. Let $\alpha(t) = (\rho C_{D_{\text{airbrakes}}} A(t))/2$:

$$I[v_0, A(t)] = \int_{v_0}^0 \frac{(\alpha_0 + c)\dot{x}^3}{(\alpha_0 + c)\frac{\dot{x}^2}{m} + g} d\dot{x} \quad (11.19)$$

We choose a constant $A(t) = A_0$ (which implies $\alpha = \alpha_0 = (\rho C_{D_{\text{airbrakes}}} A_0)/2$) as from the feedforward, which simplifies the functional to a simple function, of the form $I(v_0, A_0)$. Integration gives

$$I(v_0, A_0) = \frac{m(\dot{x}^2(\alpha_0 + c) - gm \ln(\dot{x}^2(\alpha_0 + c) + gam)}{2(\alpha_0 + c)} \Big|_{v_0}^0 \quad (11.20)$$

$$= \frac{m}{2} \cdot gm \frac{-\ln(gm)}{\alpha_0 + c} - \frac{m}{2} \left(v_0^2 - gm \frac{\ln(v_0^2(\alpha_0 + c) + gm)}{\alpha_0 + c} \right) \quad (11.21)$$

The energy balance becomes

$$h_f = h_0 + \frac{1}{2} \frac{v_0^2}{g} + \frac{I(v_0, A)|_{v_0}^0}{mg} \quad (11.22)$$

Upon simplification, we obtain

$$\boxed{h_f(h_0, v_0, \alpha_0) = h_0 + \frac{m}{2(\alpha_0 + c)} \ln \left(v_0^2 \cdot \frac{\alpha_0 + c}{gm} + 1 \right)} \quad (11.23)$$

The predicted height without airbrakes is given by this expression for $\alpha_0 = 0$. Note that we have again ignored the ramp phase, as this complicates the expression unnecessarily, as with the feedforward calculation.

We have unfortunately found that this method is not very accurate with the OpenRocket-calculated $C_{D_{\text{rocket}}}$; as such, we have decided to introduce a “fudge factor” for the C_D of the rocket, of the form

$$\text{rocketCd} \longrightarrow \text{rocketCd} * \text{cFudge} \quad (11.24)$$

where

$$\text{cFudge} = 8.99999871 \times 10^{-6} \cdot h_{\text{target}}^2 - 1.10354984 \times 10^{-1} \cdot h_{\text{target}} + 3.38925325 \times 10^2 \quad (11.25)$$

if h_{target} is expressed in meters. These coefficients were obtained from fitting a quadratic to tuned fudging coefficients over several target altitudes. This fudge factor appears to account for lack of taking turbulences, linear drag terms, and non-vertical motion into account. This fudge factor is calculated as soon as the target altitude h_{target} is decided upon.

We now wish to control the error $e \equiv h_{\text{target}} - h_f$ to aim for zero. We propose the following block diagram:

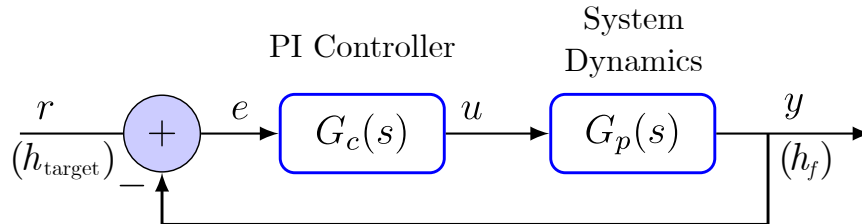


Figure 11.2. Block Diagram for the proposed Airbrakes CL controller.

We first must find the system dynamics transfer function G_p .

We linearize about the point $\alpha^* \leftrightarrow A^*$, the output of the feedforward algorithm in the previous section.

$$h_{\text{target}} \approx h_f(\alpha^*) = h_0 + \frac{m}{2(\alpha^* + c)} \ln \left(v_0^2 \cdot \frac{\alpha^* + c}{gm} + 1 \right) \quad (11.26)$$

$$\left. \frac{\partial h_f}{\partial \alpha} \right|_{\alpha^*} = \frac{m}{2} \left(-\frac{\ln \left(\frac{v_0^2}{gm}(c + \alpha^*) + 1 \right)}{(c + \alpha^*)^2} + \frac{v_0^2}{gm(c + \alpha^*) \left(\frac{v_0^2}{gm}(c + \alpha^*) + 1 \right)} \right) \quad (11.27)$$

$$\Rightarrow h_f(\alpha) \approx h_f(\alpha^*) + (\alpha - \alpha^*) \left. \frac{\partial h_f}{\partial \alpha} \right|_{\alpha^*} \quad (11.28)$$

Let $\Delta A \equiv A - A^*$, and let $\Delta h_f = h_f - h_{\text{target}}$. The transfer function is thus

$$\boxed{\frac{\Delta h_f}{\Delta A} = \left. \frac{\partial h_f}{\partial A} \right|_{\alpha^*} = \left. \frac{\partial h_f}{\partial \alpha} \right|_{\alpha^*} \frac{\partial \alpha}{\partial A} = \frac{m}{2} \left(-\frac{\ln \left(\frac{v_0^2}{gm}(c + \alpha^*) + 1 \right)}{(c + \alpha^*)^2} + \frac{v_0^2}{gm(c + \alpha^*) \left(\frac{v_0^2}{gm}(c + \alpha^*) + 1 \right)} \right) \frac{\rho C_D}{2}} \quad (11.29)$$

We calculate α^* from the feedforward A^* in the previous section.

We now propose the control scheme. An integrator is useful in removing steady state error, accumulated from a malperforming physical system. Let K be

$$K = \left. \frac{\partial h_f}{\partial A} \right|_{\alpha^*} \quad (11.30)$$

Thus,

$$\Delta h_f = K \Delta A \Rightarrow \Delta A = \frac{\Delta h_f}{K} \quad (11.31)$$

For a PI controller, we add one pole, such that

$$\Delta A = K_p e + K_i \int_0^t e \, dt \Rightarrow 0 \approx \Delta h_f = K \left(K_p e + K_i \int_0^t e \, dt \right) \quad (11.32)$$

We take the derivative:

$$0 = K K_p \dot{e} + K K_i e = K K_p \partial_t \Delta h_f + K K_i \Delta h_f \Rightarrow \partial_t \Delta h_f = -\frac{K K_i}{1 + K K_p} \Delta h_f \quad (11.33)$$

In frequency space, we have

$$s = -\frac{K K_i}{1 + K K_p} \quad (11.34)$$

In time, we have a time constant of τ if the pole location is at $s = -1/\tau$. Thus,

$$\frac{K K_i}{1 + K K_p} = \frac{1}{\tau} \quad (11.35)$$

Let $\beta = K K_p$. Thus,

$$K_p = \frac{\beta}{K} \quad (11.36)$$

As a function of τ , the integrator gain K_i is

$$K_i = \frac{1 + \beta}{K \tau} \quad (11.37)$$

We have tuned these gains (starting from $\tau = \beta = 1$) to

$$\boxed{K_p = \frac{2.5}{K}} \quad ; \quad \boxed{K_i = \frac{1}{K}} \quad (11.38)$$

which corresponds to $\beta = 2.5$ and $\tau = 3.5$ s, exactly an overdamped behavior we desire (since it is very difficult (in fact more and more difficult over time) to recover an altitude loss). These were tuned with OpenRocket RK6 simulation, over several “airbrakes malfunction factors” which related commanded exposed area to actual (simulated) exposed area.

In continuous time, the control step is

$$A(t) = A^* + K_p(h_f(t) - h_{\text{target}}) + K_i \int_0^t (h_f(t) - h_{\text{target}}) dt \quad (11.39)$$

The flight computer runs in discrete time, of course. In order to prevent windup, we implement this as

$$A[k] = A^* + K_p(h_f[k] - h_{\text{target}}) + K_i \begin{cases} I[k] & \text{if } A[k] \geq 1 \text{ and } \Delta h_f[k] > 0 \\ I[k] & \text{if } A[k] \leq 0 \text{ and } \Delta h_f[k] < 0 \\ I[k] + (h_f[k] - h_{\text{target}}) & \text{otherwise} \end{cases} \quad (11.40)$$

This control portion is only applied in the “plateau” phase (see below), after the feedforward estimate is completed.

11.3 Onboard Algorithm

We have implemented the Airbrakes Control Algorithm using a state machine with the following states (encoded in `AirbrakesControllerState`):

- `DISABLED`
- `PREPROCESS`
- `CONTROLLING_RAMP`
- `DONE`
- `PREP`
- `WAIT_FOR_START`
- `CONTROLLING_PLATEAU`
- `INFEASIBLE`

Flow from one state to the next is governed according to the following diagram:

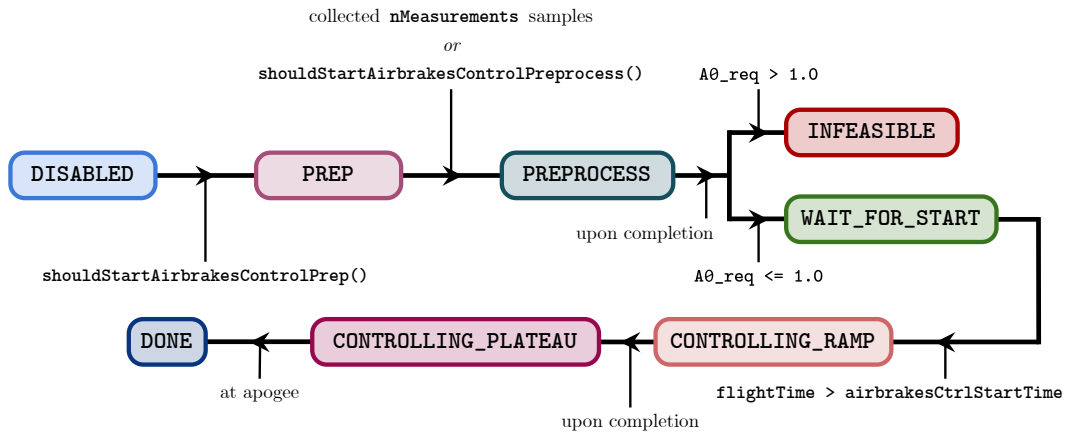


Figure 11.3. Airbrakes Controlled State Flow.

We detail the exact procedure.

- `DISABLED`

The Airbrakes controller starts in this state. No action is taken until the method `shouldStartAirbrakesControlPrep()` returns `true`, namely

```

1  return (getFlightTime() > EARLIEST_AIRBRAKES_PREP_TIME) &&
2      (!apogeeReached) && (currentRocketVel < START_AIRBRAKES_PREP_VEL)

```


- **PREP**

The **PREP** stage consists of sampling velocity $\dot{x}(t)$ and acceleration $\ddot{x}(t)$ a total of **nMeasurements** times, with timestamps saved as well. Should the **START_AIRBRAKES_PREPROC_TIME** marker have arrived prior to the full sample acquisition, the controller moves on to the **PREPROCESS** state.

- **PREPROCESS**

The **PREPROCESS** stage consists of choosing the apogee time from the list of possibilities, according to the R^2 correlation coefficient of the quintic fit to acceleration (y_i points). The quintic fit is conducted by linear least squares, according to

$$\tilde{a} = \frac{\sum_i (y_i + g)(t_i - t_{\text{apog}})^5}{\sum_i (t_i - t_{\text{apog}})^{10}} \quad (11.41)$$

and

$$R^2 = 1 - \frac{\sum_i y_i - f(x_i; \tilde{a})}{\sum_i (y_i - \bar{y})^2} \quad (11.42)$$

The apogee time t_{apog} is decided to be $\tilde{t}_{\text{apog}} + \text{AIRBRAKES_T_APOG_FUDGEDIFF}$. Next, the velocity is fitted to

$$\dot{x}(t) = a(t - t_{\text{apog}})^3 + b(t - t_{\text{apog}})^2 - g(t - t_{\text{apog}}) \quad (11.43)$$

This is again performed with linear least squares. The $X^T X$ matrix is

$$X^T X = \begin{bmatrix} \sum_i (t_i - t_{\text{apog}})^6 & \sum_i (t_i - t_{\text{apog}})^5 \\ \sum_i (t_i - t_{\text{apog}})^5 & \sum_i (t_i - t_{\text{apog}})^4 \end{bmatrix} \quad (11.44)$$

The $X^T \mathbf{y}$ matrix given by

$$X^T \mathbf{y} = \begin{bmatrix} \sum_i (t_i - t_{\text{apog}})^3 (\dot{x}_i + g(t_i - t_{\text{apog}})) \\ \sum_i (t_i - t_{\text{apog}})^2 (\dot{x}_i + g(t_i - t_{\text{apog}})) \end{bmatrix} \quad (11.45)$$

The two parameters $\tilde{a}$ and $\tilde{b}$ are thus

$$\begin{bmatrix} \tilde{a} \\ \tilde{b} \end{bmatrix} = (X^T X)^{-1} X^T \mathbf{y} \quad (11.46)$$

Next, the apogee altitude is obtained from

$$x_0 = x(t_0) - \frac{a}{4}(t_0 - t_{\text{apog}})^4 + \frac{b}{3}(t_0 - t_{\text{apog}})^3 - \frac{g}{2}(t_0 - t_{\text{apog}})^2 \quad (11.47)$$

The desired Δx is obtained from the apogee prediction and the desired altitude, either obtained by rounding:

$$x_{\text{desired}} = \left\lfloor \frac{x_0}{n} \right\rfloor n \quad (11.48)$$

where n is some integer detailing the up-to-the-closest quantity to round to, or by a hardcoded value in the flight computer code. As of **FEB 17, 2026**, the target altitude is:

6275 m

The airbrakes start time is

$$\text{airbrakesCtrlStartTime} = \text{currentTime} + \text{AIRBRAKES_TIME_DELAY} \quad (11.49)$$

The airbrakes area required is

$$A_0 = \text{reqDeployedAreaAirbrakes}(\text{currentTime} + \text{AIRBRAKES_TIME_DELAY}, \Delta x) \quad (11.50)$$

Should $A_0 > 1$, the state progresses to INFEASIBLE. Otherwise, the required A_0 is stored, and the state progresses to WAIT_FOR_START. The fudge factor cFudge is now calculated according to

$$\text{cFudge} = 8.99999871 \times 10^{-6} \cdot h_{\text{target}}^2 - 1.10354984 \times 10^{-1} \cdot h_{\text{target}} + 3.38925325 \times 10^2 \quad (11.51)$$

- WAIT_FOR_START

Simply wait for the flight time to be past `airbrakesCtrlStartTime`. The state progresses to CONTROLLING_RAMP.

- CONTROLLING_RAMP

The airbrakes area $A(t)$ here is given by

$$A(t) = 2A_0(t - \text{airbrakesCtrlStartTime}) \quad (11.52)$$

The state ends after 0.5 seconds, progressing to CONTROLLING_PLATEAU.

- CONTROLLING_PLATEAU

The airbrakes area $A(t)$ here is controlled using the algorithm detailed in the previous section. Concretely:

1. The airbrakes current area $A[k]$ is obtained, stored as `lastA` (units are m^2).
2. The ΔA is computed as $A^* - A[k] = A_0 - A[k]$, stored as `lastDeltaA`.
3. The projected altitude $h_f[k]$ is computed from the current altitude and velocity, obtained from onboard sensors. If the patching altitude has not yet been set, it is calculated and applied (this occurs once, at the first iteration in this state).
4. The Δh_f is computed as $h_f[k] - h_{\text{target}}$, stored as `lastDeltaH`.
5. The integral term is calculated:

```

1 float IofKplus1 = 0.0f;
2 if (((lastA >= 1.0f) && (lastDeltaH >= 0.0f)) || (lastA <= 1e-5f) && (lastDeltaH < 0.0f))
3 {
4   IofKplus1 = lastI;
5 } else {
6   IofKplus1 = lastI + 2.0f / K * lastDeltaH;
7 }
8 lastI = IofKplus1;
```

6. The next airbrakes area adjustment $\Delta A[k+1]$ is calculated as

$$\Delta A[k+1] = K_p * \text{lastDeltaH} + K_i * \text{lastI} \quad (11.53)$$

and thus

$$A[k+1] = \Delta A[k+1] + A^* \quad (11.54)$$

7. The airbrakes are set to the corresponding fraction:

```
1 setAirbrakesServo(nextA / a_max); // a_max = 0.0066 m^2
```

8. Should apogee have been detected, the state progresses to DONE.

- DONE

Nothing occurs in this state.

- INFEASIBLE

Nothing occurs in this state.

The code that executes this state machine can be found at

https://github.com/MIT-Rocket-Team/RT_Arduino/blob/main/Zephyrus/airbrakes/airbrakes/airbrakes.ino

Values of Constants ⟨LAST UPDATED FEB 17⟩

1 double	EARLIEST_AIRBRAKES_PREP_TIME	= 4.0;	9 double	fudge_factor	= 4.22;
2 double	START_AIRBRAKES_PREP_VEL	= 400.0;	10 double	fudge_factor_2	= 4.7;
3 double	START_AIRBRAKES_PREPROC_TIME	= 12.0;	11 double	factorK	= 0.5;
4 double	AIRBRAKES_TIME_DELAY	= 1.0;	12 double	kp_factor	= 2.5;
5 int	AIRBRAKES_N_MEASUREMENTS	= 13;	13 double	ki_factor	= 0.5;
6 int	AIRBRAKES_MEASUREMENT_FREQ_HZ	= 5;	14 int	roundToHowMuch	= 500;
7 int	AIRBRAKES_SIMULATION_T_APOG	= 33;	15 double	AIRBRAKES_MAX_AREA	= 0.0066;
8 double	AIRBRAKES_T_APOG_FUDGEDIFF	= 1.5;			

Notes from Xanthus Launch

- * Possibly detect flight from remote rocket without human assistance
 - * FLIGHT -> FLIGHT, APOGEE will be detected asap anyway
 - * PREFLIGHT -> PREFLIGHT, ensure FLIGHT detection would work from in-flight
 - * MASTER PHYSICAL SWITCH: enable/disable set_state_from_SD_card on the rocket
 - * GROUND STATION SWITCH: if enabled, allows for <something>

12 Full FC Code Printout

Airbrakes Controller

```

1  /// C Arduino Airbrakes Controller Code
2  // MARC D. NICHITIU irta
3  // MIT RT SIMULATIONS JAN 2026
4
5
6
7  #include <math.h>
8  #include <stdint.h>
9
10 HardwareSerial HWSerial(PA10, PA9);
11
12 /* ----- Compile-time constants ----- */
13 #define AIRBRAKES_N_MEASUREMENTS 13
14 #define AIRBRAKES_MEASUREMENT_FREQ_HZ 2
15 #define AIRBRAKES_SIMULATION_T_APOG 34.0f
16 #define DEBUG_AIRBRAKES_ON 0
17 #define LOOP_FREQ 100
18 #define AIRBRAKES_START_TIME 13.0f // seconds
19
20 /* ----- Measurements types ----- */
21 typedef struct {
22     float velocityMeasurement;
23     float timeStamp;
24 } AirbrakesVelocityMeasurement;
25
26 typedef struct {
27     float accelerationMeasurement;
28     float timeStamp;
29 } AirbrakesAccelerationMeasurement;
30
31 /* Constructors */
32 AirbrakesVelocityMeasurement AirbrakesVelocityMeasurement_init(float ts, float vm) {
33     AirbrakesVelocityMeasurement m;
34     m.timeStamp = ts;
35     m.velocityMeasurement = vm;
36     return m;
37 }
38
39 AirbrakesAccelerationMeasurement AirbrakesAccelerationMeasurement_init(float ts, float am) {
40     AirbrakesAccelerationMeasurement m;
41     m.timeStamp = ts;
42     m.accelerationMeasurement = am;
43     return m;
44 }
45
46 HardwareTimer *myTim = new HardwareTimer(TIM1);
47
48 /* ----- RocketStatus placeholder ----- */
49 typedef struct {
50     float altitude; // m
51     float vel_z; // m/s
52     float accel_z; // m/s^2
53     bool apogeeReached;
54 } RocketStatus;
55
56 /* ----- Controller state ----- */
57 typedef enum {
58     DISABLED = 0,
59     PREP,
60     PREPROCESS,
61     WAIT_FOR_START,
62     CONTROLLING_RAMP,
63     CONTROLLING_PLATEAU,
64     DONE,
65     INFEASIBLE
66 } AirbrakesControllerState;
67
68 /* ----- Globals ----- */

```

```
69 AirbrakesControllerState state = DISABLED;
70
71 const float g = 9.81f;
72
73 float mass = 39.140453380154554;
74 float rho = 0.736115423712237;
75 float airbrakesCd = 1.28f;
76 float rocketCd = 0.5225681679119347f;
77 float a_ref = 0.019289796351014733f;
78 float a_max = 0.0066f;
79 float fudge_factor = 3.2f;
80 float fudge_factor_2 = 3.5f;
81
82 float EARLIEST_AIRBRAKES_PREP_TIME = 4.0f;
83 float START_AIRBRAKES_PREP_VEL = 400.0f;
84 float START_AIRBRAKES_PREPROC_TIME = 12.5f;
85 float AIRBRAKES_TIME_DELAY = 1.0f;
86 float AIRBRAKES_T_APOG_FUDGEDIFF = 1.5f;
87
88 int roundToHowMuch = 100;
89
90 float t_apog = 35.5f;
91 float coeffA = -0.0154397511f;
92 float coeffB = -0.3379534959f;
93
94 float alt0 = 0.0f;
95 float predictedAlt = 0.0f;
96 float desiredDeltaX = 0.0f;
97
98 float airbrakesCtrlStartTime = 1e10f;
99 float A0_req = 0.0f;
100
101
102 float Astar = 0.0f;
103 float patchingAltitude = 0.0f;
104 float velContribFudge = 1.0f;
105 float cFudge = 0.825f;
106 float K = 1;
107 float factorK = 0.5f; // not actually a fudge factor.
108 float kp_factor = 2.5f;
109 float ki_factor = 0.5f;
110
111 float lastA = 0;
112 float lastDeltaA = 0;
113 float lastDeltaH = 0;
114 float lastHf = 0;
115 float lastI = 0;
116
117
118 /* live rocket values !!! REPLACE IN LOOP W TELEMETRY */
119 float currentRocketVel = 0.0f;
120 float currentRocketAccel = 0.0f;
121 bool apogeeReached = false;
122 float currentRocketAlt = 1.0f;
123 float targetAlt = 6275.0f;
124
125 float lastTimeStamp = 0;
126
127 /* data collection */
128 AirbrakesAccelerationMeasurement accelData[AIRBRAKES_N_MEASUREMENTS];
129 AirbrakesVelocityMeasurement velData[AIRBRAKES_N_MEASUREMENTS];
130
131 /* counters */
132 int datIndex = 0;
133 int counter = 0;
134
135 unsigned long lastMeasurement = millis();
136
137 unsigned long startTime = millis();
138 bool simEnabled = false;
139
140 float globalDP;
```

```

141
142 /* ----- Helpers (declare before use) ----- */
143
144
145 /* Fudge for Coeff of Drag from OpenRocket */
146 float getCfudge(float alt) {
147     return 8.99999871e-06f * alt * alt - 1.10354984e-01f * alt + 3.38925325e+02f;
148 }
149
150 float maxf(float a, float b) {
151     return (a > b) ? a : b;
152 }
153
154 /* Fast integer-power helpers */
155 float p4(float x) {
156     float x2 = x * x;
157     return x2 * x2;
158 }
159 float p5(float x) {
160     return p4(x) * x;
161 }
162 float p6(float x) {
163     float x3 = x * x * x;
164     return x3 * x3;
165 }
166 float p7(float x) {
167     return p6(x) * x;
168 }
169 float p8(float x) {
170     float x4 = p4(x);
171     return x4 * x4;
172 }
173 float p9(float x) {
174     return p8(x) * x;
175 }
176 float p10(float x) {
177     float x5 = p5(x);
178     return x5 * x5;
179 }
180
181 float pow5f_fast(float x) {
182     return p5(x);
183 }
184 float pow10f_fast(float x) {
185     return p10(x);
186 }
187
188 /* Avoid Arduino macro collision with sq() by using different names */
189 float sqf_local(float x) {
190     return x * x;
191 }
192 float cubef_local(float x) {
193     return x * x * x;
194 }
195 float pow4f_local(float x) {
196     float x2 = x * x;
197     return x2 * x2;
198 }
199
200 bool inverse2x2Matrix(const float A[2][2], float Ainv[2][2]) {
201     float a = A[0][0], b = A[0][1];
202     float c = A[1][0], d = A[1][1];
203     float det = a * d - b * c;
204
205     if (fabsf(det) <= 1e-6f) {
206         Ainv[0][0] = 0.0f;
207         Ainv[0][1] = 0.0f;
208         Ainv[1][0] = 0.0f;
209         Ainv[1][1] = 0.0f;
210         return false;
211     }
212

```

```

213 float f = 1.0f / det;
214 Ainv[0][0] = f * d;
215 Ainv[0][1] = -f * c;
216 Ainv[1][0] = -f * b;
217 Ainv[1][1] = f * a;
218 return true;
219 }
220
221 /* Flight time in seconds */
222 float getFlightTime() {
223     if (!simEnabled) {
224         return 0;
225     }
226     return (millis() - startTime) * 0.001f; // FIXED (seconds)
227 }
228
229 /* actuator command */
230 void setAirbrakesServo(float deployedFraction) {
231     if (deployedFraction < 0.0f) deployedFraction = 0.0f;
232     if (deployedFraction > 1.0f) deployedFraction = 1.0f;
233     HWSerial.println(deployedFraction);
234     globalDP = 1-deployedFraction;
235 }
236
237 /* actuator reading */
238 float getAirbrakesServo() {
239     return globalDP;
240 }
241
242 /* models */
243 float accelModel(float t, float a, float custom_t_apog) {
244     float dt = t - custom_t_apog;
245     return a * pow5f_fast(dt) - g;
246 }
247
248 float getR2fromFit_accel(const AirbrakesAccelerationMeasurement *data,
249                          size_t n,
250                          float a,
251                          float custom_t_apog) {
252     if (!data || n == 0) return 0.0f;
253
254     float ss_res = 0.0f;
255     float ss_tot = 0.0f;
256     float sum_y = 0.0f;
257
258     for (size_t i = 0; i < n; i++) {
259         float y = data[i].accelerationMeasurement;
260         float yh = accelModel(data[i].timeStamp, a, custom_t_apog);
261         float r = y - yh;
262         ss_res += r * r;
263         sum_y += y;
264     }
265
266     float mean = sum_y / (float)n;
267     for (size_t i = 0; i < n; i++) {
268         float d = data[i].accelerationMeasurement - mean;
269         ss_tot += d * d;
270     }
271
272     if (ss_tot == 0.0f) return 0.0f;
273     return 1.0f - (ss_res / ss_tot);
274 }
275
276 int argmax(const float *arr, size_t n) {
277     if (!arr || n == 0) return -1;
278     float best = arr[0];
279     int idx = 0;
280     for (size_t i = 1; i < n; i++) {
281         if (arr[i] > best) {
282             best = arr[i];
283             idx = (int)i;
284         }

```



```

285     }
286     return idx;
287 }
288
289 /* printing helper */
290 void printFloatArray3(const char *label, const float a[3]) {
291     HWSerial.print(label);
292     HWSerial.print("[");
293     HWSerial.print(a[0], 4);
294     HWSerial.print(", ");
295     HWSerial.print(a[1], 4);
296     HWSerial.print(", ");
297     HWSerial.print(a[2], 4);
298     HWSerial.println("]");
299 }
300
301 /* calculate the area needed for airbrakes */
302 float reqDeployedAreaAirbrakes(float t_0, float deltaX) {
303     float a = coeffA;
304     float b = coeffB;
305     float dt = (t_0 - t_apog);
306
307     float a2 = a * a;
308     float a3 = a2 * a;
309     float b2 = b * b;
310     float b3 = b2 * b;
311     float g2 = g * g;
312     float g3 = g2 * g;
313
314     float term =
315         (a3)*p10(dt) / 10.0f
316         + (a2 * b) * p9(dt) / 3.0f
317         + (3.0f * a * b2 - 3.0f * a2 * g) * p8(dt) / 8.0f
318         + (b3 - 6.0f * a * b * g) * p7(dt) / 7.0f
319         + (a * g2 - b2 * g) * p6(dt) / 2.0f
320         + (3.0f * b * g2) * p5(dt) / 5.0f
321         - (g3)*p4(dt) / 4.0f;
322
323     float xi = -term;
324
325     float local_fudge = (deltaX > 40.0f) ? fudge_factor : fudge_factor_2;
326
327     float denom = airbrakesCd * rho * xi;
328     if (denom == 0.0f) return 0.0f;
329     if (a_max == 0.0f) return 0.0f;
330
331     float a_0 = local_fudge * 2.0f * mass * g * deltaX / denom;
332     return maxf(0.0f, a_0 / a_max);
333 }
334
335 /* estimates */
336 float getVelocityEstimate(float t) {
337     float dt = t - t_apog;
338     return coeffA * cubef_local(dt)
339         + coeffB * sqf_local(dt)
340         - g * dt;
341 }
342
343 float getAltitudeEstimate(float t, float alt0_local) {
344     float dt = t - t_apog;
345     return coeffA * pow4f_local(dt) / 4.0f
346         + coeffB * cubef_local(dt) / 3.0f
347         - g * sqf_local(dt) / 2.0f
348         + alt0_local;
349 }
350
351 float getAltitudeEstimate(float t) {
352     return getAltitudeEstimate(t, alt0);
353 }
354
355 /* start conditions */
356 bool shouldStartAirbrakesControlPrep() {

```

```

357   HWSerial.print("Flight time: ");
358   HWSerial.println(getFlightTime());
359   HWSerial.print("Current Vel: ");
360   HWSerial.println(currentRocketVel);
361   return (getFlightTime() > EARLIEST_AIRBRAKES_PREP_TIME) && (!apogeeReached) && (currentRocketVel
    < START_AIRBRAKES_PREP_VEL);
362 }
363
364 bool shouldStartAirbrakesControlPreprocess() {
365     return (getFlightTime() > START_AIRBRAKES_PREPROC_TIME) && (!apogeeReached);
366 }
367
368
369 /* Conrad Altitude Estimate */
370 float computeFinalAltitude_Conrad(float A, float h0, float v0) {
371     const float fudge_factor_conrad = 3.2f;
372     const float fudged_alt_diff = 13.0f;
373
374     float m = mass;
375     float c = rho * rocketCd * a_ref / 2.0f;
376     c *= cFudge;
377     float alpha = rho * airbrakesCd * A / 2.0f;
378
379     // Fudging
380     alpha /= fudge_factor_conrad;
381
382     float hf = h0 + velContribFudge * m / (2.0 * (alpha + c)) * log((v0 * v0 * (alpha + c)) / g / m +
    1);
383     return hf - fudged_alt_diff + patchingAltitude;
384 }
385
386 /* Compute Coefficient for control */
387 float computeK(float Astar, float h0, float v0) {
388     const float fudge_factor_conrad = 3.2f;
389     const float fudged_alt_diff = 13.0f;
390
391     float m = mass;
392     float c = rho * rocketCd * a_ref / 2.0f;
393     float alphaStar = rho * airbrakesCd * Astar / 2.0f;
394
395     // Fudging
396     alphaStar /= fudge_factor_conrad;
397
398     float K0 = m / 2 * (-1 / (c + alphaStar) / (c + alphaStar) * log(v0 * v0 / g / m * (c + alphaStar
    ) + 1) + 1 / (c + alphaStar) * v0 * v0 / g / m / (v0 * v0 / g / m * (c + alphaStar) + 1));
399
400     return K0;
401 }
402
403
404 void handleAirbrakesState() {
405     RocketStatus status;
406     status.altitude = 0.0f;
407     status.vel_z = currentRocketVel;
408     status.accel_z = currentRocketAccel;
409     status.apogeeReached = apogeeReached;
410
411     float currentTime = getFlightTime();
412
413     apogeeReached = status.apogeeReached;
414     currentRocketVel = status.vel_z;
415     currentRocketAccel = status.accel_z;
416
417     const int everyHowMany = 1000 / AIRBRAKES_MEASUREMENT_FREQ_HZ;
418     const int nMeasurements = AIRBRAKES_N_MEASUREMENTS;
419
420     // DISABLED : awaiting start of prep (data collection)
421     if (state == DISABLED) {
422         setAirbrakesServo(0.0f);
423         //HWSerial.print("[Airbrakes] Status:  DISABLED                \r");
424         state = shouldStartAirbrakesControlPrep() ? PREP : DISABLED;
425         if (state == PREP) {

```

```

426     datIndex = 0;
427     counter = 0;
428     HWSerial.println("[Airbrakes] Entering PREP");
429     HWSerial.print("[Airbrakes] Current Time: ");
430     HWSerial.println(currentTime, 4);
431 }
432 }
433
434 else if (state == PREP) {
435     if ((millis() - lastMeasurement > (1000 / AIRBRAKES_MEASUREMENT_FREQ_HZ))) {
436         lastMeasurement = millis();
437         if (datIndex < nMeasurements) {
438             accelData[datIndex] = AirbrakesAccelerationMeasurement_init(currentTime, currentRocketAccel
439 );
440             velData[datIndex] = AirbrakesVelocityMeasurement_init(currentTime, currentRocketVel);
441             datIndex++;
442         }
443     }
444     counter++;
445
446     if (datIndex >= nMeasurements) {
447         state = PREPROCESS;
448         counter = 0;
449
450         for (int i = 0; i < nMeasurements; i++) {
451             HWSerial.print("[Airbrakes] Data ");
452             HWSerial.print(i);
453             HWSerial.print(": Vel=");
454             HWSerial.print(velData[i].velocityMeasurement, 4);
455             HWSerial.print(" @ ");
456             HWSerial.print(velData[i].timeStamp, 4);
457             HWSerial.print(" | Accel=");
458             HWSerial.print(accelData[i].accelerationMeasurement, 4);
459             HWSerial.print(" @ ");
460             HWSerial.println(accelData[i].timeStamp, 4);
461         }
462
463         HWSerial.println("[Airbrakes] Entering PREPROCESS (data collected)");
464         HWSerial.print("[Airbrakes] Current Time: ");
465         HWSerial.println(currentTime, 4);
466     } else {
467         state = shouldStartAirbrakesControlPreprocess() ? PREPROCESS : PREP;
468         if (state == PREPROCESS) {
469
470             for (int i = 0; i < nMeasurements; i++) {
471                 HWSerial.print("[Airbrakes] Data ");
472                 HWSerial.print(i);
473                 HWSerial.print(": Vel=");
474                 HWSerial.print(velData[i].velocityMeasurement, 4);
475                 HWSerial.print(" @ ");
476                 HWSerial.print(velData[i].timeStamp, 4);
477                 HWSerial.print(" | Accel=");
478                 HWSerial.print(accelData[i].accelerationMeasurement, 4);
479                 HWSerial.print(" @ ");
480                 HWSerial.println(accelData[i].timeStamp, 4);
481             }
482
483             counter = 0;
484             HWSerial.println("[Airbrakes] Entering PREPROCESS (time's up)");
485             HWSerial.print("[Airbrakes] Current Time: ");
486             HWSerial.println(currentTime, 4);
487         }
488     }
489 }
490
491 // PREPROCESS: Predict Apogee, t_apog, and A0_req
492 else if (state == PREPROCESS) {
493     const float t_apog_trials[3] = {
494         AIRBRAKES_SIMULATION_T_APOG - 1.0f,
495         AIRBRAKES_SIMULATION_T_APOG,
496         AIRBRAKES_SIMULATION_T_APOG + 1.0f

```

```

497     };
498
499     float resulting_R2_values[3] = { 0.0f, 0.0f, 0.0f };
500
501     for (size_t i = 0; i < 3; i++) {
502         float sum_num = 0.0f;
503         float sum_den = 0.0f;
504
505         for (size_t j = 0; j < (size_t)nMeasurements; j++) {
506             float dt = accelData[j].timeStamp - t_apog_trials[i];
507             sum_den += pow10f_fast(dt);
508             sum_num += (accelData[j].accelerationMeasurement + g) * pow5f_fast(dt);
509         }
510
511         float a_coeff = (sum_den != 0.0f) ? (sum_num / sum_den) : 0.0f;
512         resulting_R2_values[i] = getR2fromFit_accel(accelData, (size_t)nMeasurements, a_coeff,
513         t_apog_trials[i]);
514     }
515
516     printFloatArray3("[Airbrakes] t_apog trials: ", t_apog_trials);
517     printFloatArray3("[Airbrakes] R2 values:      ", resulting_R2_values);
518
519     int best_idx = argmax(resulting_R2_values, 3);
520     float best_t = (best_idx >= 0) ? t_apog_trials[(size_t)best_idx] : AIRBRAKES_SIMULATION_T_APOG;
521     t_apog = best_t + AIRBRAKES_T_APOG_FUDGEDIFF;
522
523     HWSerial.print("[Airbrakes] Choosing t_apog = ");
524     HWSerial.println(t_apog, 4);
525
526     // Fit velocity coefficients
527     float XT_X[2][2] = { { 0.0f, 0.0f }, { 0.0f, 0.0f } };
528     float XT_y[2] = { 0.0f, 0.0f };
529
530     for (int i = 0; i < nMeasurements; i++) {
531         float dt = velData[i].timeStamp - t_apog;
532         float dt2 = dt * dt;
533         float dt3 = dt2 * dt;
534         float dt4 = dt2 * dt2;
535         float dt5 = dt4 * dt;
536         float dt6 = dt3 * dt3;
537
538         XT_X[0][0] += dt6;
539         XT_X[0][1] += dt5;
540         XT_X[1][0] += dt5;
541         XT_X[1][1] += dt4;
542
543         float vi = velData[i].velocityMeasurement;
544         float yi = vi + g * dt;
545
546         XT_y[0] += dt3 * yi;
547         XT_y[1] += dt2 * yi;
548     }
549
550     float XT_X_inv[2][2];
551     if (!inverse2x2Matrix(XT_X, XT_X_inv)) {
552         HWSerial.println("[Airbrakes] XT_X singular -> INFEASIBLE");
553         state = INFEASIBLE;
554         return;
555     }
556
557     coeffA = XT_X_inv[0][0] * XT_y[0] + XT_X_inv[0][1] * XT_y[1];
558     coeffB = XT_X_inv[1][0] * XT_y[0] + XT_X_inv[1][1] * XT_y[1];
559
560     HWSerial.print("[Airbrakes] Velocity fit: a=");
561     HWSerial.print(coeffA, 6);
562     HWSerial.print(" b=");
563     HWSerial.println(coeffB, 6);
564     HWSerial.print("[Airbrakes] Current Altitude: ");
565     HWSerial.print(currentRocketAlt, 6);
566     HWSerial.print(" @ ");
567     HWSerial.println(getFlightTime(), 6);
568     alt0 = currentRocketAlt - getAltitudeEstimate(getFlightTime());

```

```

568     predictedAlt = getAltitudeEstimate(t_apog);
569
570     float desiredAlt = 6275.0f; //floorf(predictedAlt / (float)roundToHowMuch) * (float)
roundToHowMuch;
571     targetAlt = desiredAlt;
572     desiredDeltaX = predictedAlt - desiredAlt;
573     cFudge = getCfudge(desiredAlt);
574     HWSerial.print("[Airbrakes] Predicted Altitude: ");
575     HWSerial.println(predictedAlt, 6);
576     HWSerial.print("[Airbrakes] Desired Altitude: ");
577     HWSerial.println(desiredAlt, 6);
578     HWSerial.print("[Airbrakes] Aiming for    X in Altitude: ");
579     HWSerial.println(desiredDeltaX, 6);
580
581
582     bool tooLate = currentTime > AIRBRAKES_START_TIME - 0.25;
583     airbrakesCtrlStartTime = tooLate ? currentTime + AIRBRAKES_TIME_DELAY : AIRBRAKES_START_TIME;
584     AO_req = reqDeployedAreaAirbrakes(airbrakesCtrlStartTime, desiredDeltaX);
585
586     Astar = AO_req * a_max;
587     lastA = Astar;
588     K = computeK(Astar, currentRocketAlt, currentRocketVel) * factorK;
589
590
591
592     if (AO_req > 1.0f) {
593         HWSerial.print("[Airbrakes] Req A=");
594         HWSerial.print(AO_req, 3);
595         HWSerial.println(" > 1.0 (infeasible)");
596         if (DEBUG_AIRBRAKES_ON) {
597             AO_req = 1.0f;
598             state = WAIT_FOR_START;
599         } else {
600             state = INFEASIBLE;
601         }
602     } else {
603         HWSerial.println("[Airbrakes] Reaching Apogee is Feasible");
604         HWSerial.print("[Airbrakes] AO_req=");
605         HWSerial.println(AO_req, 3);
606         HWSerial.print("[Airbrakes] Desired start time: ");
607         HWSerial.println(airbrakesCtrlStartTime, 3);
608         state = WAIT_FOR_START;
609     }
610 }
611
612
613 // Wait for the designated start time.
614 else if (state == WAIT_FOR_START) {
615     if (getFlightTime() >= airbrakesCtrlStartTime) {
616         state = CONTROLLING_RAMP;
617         HWSerial.println("[Airbrakes] Beginning control");
618         HWSerial.print("[Airbrakes] Current Time: ");
619         HWSerial.println(currentTime, 4);
620     }
621 }
622
623 // Ramp up
624 else if (state == CONTROLLING_RAMP) {
625     float t = getFlightTime();
626     if (t >= airbrakesCtrlStartTime) {
627         float deployedFraction = 2.0f * AO_req * (t - airbrakesCtrlStartTime);
628         setAirbrakesServo(deployedFraction);
629     }
630     if (t >= airbrakesCtrlStartTime + 0.5f) {
631         state = CONTROLLING_PLATEAU;
632         setAirbrakesServo(AO_req);
633     }
634 }
635 // plateau
636 else if (state == CONTROLLING_PLATEAU) {
637
638     float Ki = ki_factor * 2.0f / K;

```

```

639     float Kp = kp_factor / K;
640
641     lastA = getAirbrakesServo() * a_max;
642
643     lastDeltaA = lastA - Astar;
644
645     float current_hf = computeFinalAltitude_Conrad(lastA, currentRocketAlt, currentRocketVel);
646     if (patchingAltitude == 0) {
647         patchingAltitude = targetAlt - current_hf;
648         current_hf = computeFinalAltitude_Conrad(lastA, currentRocketAlt, currentRocketVel);
649     }
650     lastDeltaH = current_hf - targetAlt;
651     lastHf = current_hf;
652
653     // decide integral term
654     float IofKplus1 = 0.0f;
655     if (((lastA >= 1.0f) && (lastDeltaH >= 0.0f)) || (lastA <= 1e-5f) && (lastDeltaH < 0.0f)) {
656         IofKplus1 = lastI;
657     } else {
658         IofKplus1 = lastI + 2.0f / K * lastDeltaH;
659     }
660     lastI = IofKplus1;
661
662     float nextDeltaA = Kp * lastDeltaH + Ki * lastI;
663     float nextA = Astar + nextDeltaA;
664     setAirbrakesServo(nextA / a_max);
665
666
667     if (status.vel_z <= 0.0f || status.apogeeReached) {
668         state = DONE;
669         setAirbrakesServo(0.0f);
670     }
671 }
672 // DONE / INFEASIBLE: do nothing
673 }
674
675 uint16_t deployToUs(float dp) {
676     return 1500 + (1.0 / .15) * (dp * 80.0 - 40.0);
677 }
678
679 void Update_IT_callback() {
680     myTim->setCaptureCompare(1, deployToUs(globalDP), MICROSEC_COMPARE_FORMAT);
681 }
682
683
684
685 /* ----- Arduino entry points ----- */
686 void setup() {
687     //HWSerial.setTx(PA0);
688     //HWSerial.setRx(PA1);
689     //HWSerial.println("boi i'm starting up get hyped");
690     myTim->setMode(1, TIMER_OUTPUT_COMPARE_PWM1, PA_8);
691     myTim->setOverflow(20000, MICROSEC_FORMAT);
692     myTim->setCaptureCompare(1, deployToUs(0), MICROSEC_COMPARE_FORMAT);
693     myTim->attachInterrupt(Update_IT_callback);
694     myTim->resume();
695
696     HWSerial.begin(115200);
697     while (!HWSerial) { ; }
698     HWSerial.println("Airbrakes controller starting...");
699 }
700
701 void loop() {
702     // Replace with real telemetry
703     if (millis() - lastTimeStamp >= 1000.0f / LOOP_FREQ) {
704
705         // READ TELEMETRY
706
707         while (HWSerial.available() >= 7) {
708             if (HWSerial.peek() == 0xAA) {
709                 int n = 6; // size of the packet
710                 uint8_t data[n + 1];

```

```

711     HWSerial.readBytes((char *)data, n + 1);
712     uint8_t checksum = 0;
713     for (int i = 1; i < n; i++) {
714         checksum += data[i];
715     }
716     if (checksum == data[n]) {
717         // proc start, vel/accel data, and apogee status.
718         switch (data[1]) {
719             case 0x00:
720                 memcpy(&currentRocketVel, &data[2], 4);
721                 break;
722             case 0x01:
723                 memcpy(&currentRocketAccel, &data[2], 4);
724                 break;
725             case 0x02:
726                 memcpy(&apogeeReached, &data[2], 4);
727                 break;
728             case 0x03:
729                 simEnabled = true;
730                 startTime = millis();
731                 break;
732             case 0x04:
733                 memcpy(&currentRocketAlt, &data[2], 4);
734                 break;
735         }
736     }
737     } else {
738         HWSerial.read();
739     }
740     /*
741     HWSerial.print("VEL0: ");
742     HWSerial.println(currentRocketVel);
743     HWSerial.print("ACCEL: ");
744     HWSerial.println(currentRocketAccel);
745     HWSerial.print("APOGEE: ");
746     HWSerial.println(apogeeReached);
747     */
748 }
749
750 handleAirbrakesState();
751 lastTimeStamp = millis();
752 }
753 }

```